Implementing and Visualizing Adversarial Examples

CYSE 635 — Module 03 Lab Report

Erfan Basiri
CYSE 635

1. Introduction

Why adversarial examples matter. A classifier that is accurate on its test set can still be fooled by inputs that an attacker has changed only slightly. These *adversarial examples* are a security problem rather than a curiosity: the change can be too small for a person to notice, the attack can be computed in milliseconds, and it often works even against a model the attacker cannot see. Wherever a model's decision matters (reading checks and forms, recognizing road signs, filtering malware or spam, verifying faces), an adversary who controls part of the input can steer the decision, and accuracy on clean data says nothing about this risk. Measuring how much perturbation a model tolerates, understanding why it fails and testing defenses are therefore part of evaluating any model that will face adversaries.

Objective. In this lab I generated, evaluated and visualized adversarial examples for handwritten-digit classification on MNIST. The target is DigitNet, the CNN I built in Module 02, which classifies 99.70 % of the clean test images correctly. I implemented seven attacks in PyTorch, from random noise and FGSM to PGD, targeted PGD and two black-box attacks; measured how accuracy falls as the perturbation budget ε grows and how visible the perturbation becomes; and visualized what the attacks change in the input, in the feature maps and in the model's attention. Finally, I implemented and attacked two kinds of countermeasures: adversarial training and input preprocessing.

Key results.

- DigitNet is **not robust**. At $\varepsilon = 0.1$ (25 of 255 gray levels per pixel), FGSM reduces its accuracy from 99.70 % to 71.3 % and PGD to 12.0 %; at $\varepsilon = 0.2$ PGD fools it on every test image. Random noise of the same size barely matters (96.0 %), so the *direction* of the perturbation is what hurts. The more accurate DigitNet is also less robust than the simpler SimpleCNN.
- Inside the network, adversarial perturbations are **amplified** layer by layer (relative change 0.23 at the input, 0.87 in the final features, vs. 0.40 for random noise), and Grad-CAM shows the attention moving away from the true digit. The attacks also work as a **black box**: examples crafted on an independently trained DigitNet leave the target at 7.3 % ($\varepsilon = 0.2$).
- **PGD adversarial training** was the most effective defense: 97.3 % accuracy under PGD at $\varepsilon = 0.3$ (95.3 % with 10 restarts) for a clean accuracy of 99.49 %. Binarizing the input looked robust against a naive attacker (99.0 %) only because it hides the gradient; an adaptive BPDA attack brought it down to 64.0 %.

All code, results and figures are in the GitHub repository `https://github.com/EBARV/CYSE635-MNIST-Adversarial` (Section 6).

2. Methodology

2.1 Target Model and Comparison Models

The lab asks for a CNN with at least two convolutional layers and one fully connected layer. Instead of training a new one, I attacked **DigitNet**, the network I built and analysed in Module 02, loading the exact checkpoint reported there (seed 0). It is a compact residual CNN: a 5×5 stem convolution, three residual blocks with batch normalization and squeeze-and-excitation channel attention (seven convolutional layers in total), global average pooling and a classifier with two fully connected layers (described in detail in my Module 02 report). It reaches **99.70 %** test accuracy. Because every

model standardizes its input in its first layer, all attacks work on ordinary pixel values in $[0, 1]$. I also loaded the same *data split* as in Module 02 (55,000 training, 5,000 validation, 10,000 test images), so none of the models has seen a test image.

To see whether the findings depend on the architecture, I attacked three further models (Table 1): **SimpleCNN**, the LeNet-style baseline from Module 02 (two convolutional and two fully connected layers); the instructor's 5-layer fully connected network **TemplateDNN**, retrained exactly as in the lab template; and a second DigitNet trained with a different seed, which serves as the attacker's *surrogate* in the black-box experiments.

Table 1. Models used in this lab. All were trained on the same 55,000 images; accuracy is on the 10,000 clean test images.

Model	Parameters	Conv layers	FC layers	Clean acc. (%)	Role
DigitNet	330,726	7	2	99.70	target (Module 02 model)
SimpleCNN	454,922	2	2	99.51	Module 02 baseline
TemplateDNN	575,050	0	5	97.87	instructor's template
DigitNet (seed 1)	330,726	7	2	99.60	black-box surrogate

2.2 Threat Model

An adversarial example is an input $x' = x + \delta$ that is classified differently from x although δ is small. I measure "small" with the L_∞ norm, the standard choice for MNIST: every pixel may change by at most ε, so $\|\delta\|_\infty \le \varepsilon$, and x' must remain a valid image in $[0, 1]$. A budget of $\varepsilon = 0.1$ lets each pixel move by about 25 of 255 gray levels; $\varepsilon = 0.3$ is the largest budget commonly used for MNIST (Madry et al., 2018). I report most results at $\varepsilon \in \{0.1, 0.2, 0.3\}$ and sweep ε from 0 to 0.3 (0.4 for the defenses). Most attacks are *white-box*: the attacker knows the architecture and weights and can compute gradients. In Section 3.5 I also test two *black-box* settings, in which the attacker either has no access to the model (transfer attacks) or can only observe its output probabilities (query attacks).

2.3 Adversarial Example Generation

All attacks are implemented from scratch in PyTorch and run batched on the GPU (Listing 1). $J(\theta, x, y)$ denotes the cross-entropy loss of the model with parameters θ on image x with label y, and Π_ε is the projection onto the allowed set, i.e. clipping to $[x - \varepsilon, x + \varepsilon] \cap [0, 1]$.

- **Random noise (baseline).** $x' = \Pi_\varepsilon\big(x + \varepsilon \cdot \mathrm{sign}(r)\big)$ with $r \sim \mathcal{N}(0, I)$, i.e. a random $\pm\varepsilon$ on every pixel, as in the lab template; I also test Gaussian noise $x + \mathcal{N}(0, \varepsilon^2 I)$. The sign version has exactly the same L_∞ and L_2 size as FGSM, so it isolates the effect of the perturbation's *direction*.
- **FGSM** (Goodfellow et al., 2015): one gradient step, $x' = \Pi_\varepsilon\big(x + \varepsilon \cdot \mathrm{sign}(\nabla_x J(\theta, x, y))\big)$. It costs a single forward and backward pass.
- **PGD** (Madry et al., 2018): start from a random point in the ε-ball and repeat $x^{t+1} = \Pi_\varepsilon\big(x^t + \alpha \cdot \mathrm{sign}(\nabla_x J(\theta, x^t, y))\big)$. I use 40 steps with $\alpha = 2.5\,\varepsilon/40$ unless stated otherwise; the strongest evaluation uses 100 steps and 10 random restarts and keeps, per image, the first successful restart. Section 3.3 checks that these settings have converged.
- **MI-FGSM** (Dong et al., 2018): an iterative FGSM (10 steps of $\varepsilon/10$) that accumulates the L_1-normalized gradient with momentum $\mu = 1$ before taking its sign. The momentum makes the update direction less dependent on the particular model, and the method was designed to *transfer* better to unseen models.
- **Targeted PGD**: the same iteration, but the loss with respect to a chosen target class t is *decreased*, $x^{t+1} = \Pi_\varepsilon(x^t - \alpha \cdot \mathrm{sign}(\nabla_x J(\theta, x^t, t)))$, using 100 steps.
- **Transfer attack (black box, no queries)**: adversarial examples crafted with PGD, MI-FGSM or FGSM on a *source* model are fed to a different *target* model.
- **SimBA** (Guo et al., 2019) **(black box, score-based)**: the attacker only sees the softmax output. In a random pixel order it tries setting one pixel's perturbation to $+\varepsilon$, then $-\varepsilon$, and keeps the change whenever the probability of the true class drops. It stops when the image is misclassified, so its cost is measured in model queries (at most 2×784 per image).

2.4 Defenses

- **Adversarial training (robust optimization).** Training solves $\min_\theta \mathbb{E}_{(x,y)}\left[\max_{\|\delta\|_\infty \le \varepsilon} J(\theta, x+\delta, y)\right]$ by replacing every training batch with adversarial examples crafted on the current model. I trained two DigitNets from scratch with $\varepsilon = 0.3$: **PGD-AT** (Madry et al., 2018), with a 10-step PGD inner maximization, and **FGSM-RS** ("fast" adversarial training, Wong et al., 2020), which uses a single FGSM step of 1.25ε from a random start. Both use AdamW (weight decay 0.01), a one-cycle learning-rate schedule with peak 10^{-3}, batch size 128 and 20 epochs. ε grows linearly from 0 to 0.3 over the first 5 epochs, a common way to keep the first epochs of adversarial training on MNIST stable. The checkpoint with the best PGD-20 accuracy on 1,000 *validation* images is kept.
- **Input preprocessing.** Three transformations are applied to the input of the standard DigitNet: bit-depth reduction to 1 bit (binarization at 0.5), a 3×3 median filter and a Gaussian blur ($\sigma = 1$). I attack them in three ways: an *oblivious* attacker who crafts PGD examples on the undefended model; a *naive* white-box attacker who back-propagates through the transformation; and an *adaptive* attacker who uses BPDA (Athalye et al., 2018), i.e. the exact transformation in the forward pass and the identity in the backward pass, with 100 PGD steps.

2.5 Testing Protocol and Metrics

Each attack is evaluated on all 10,000 test images unless the table says otherwise. The expensive evaluations use the first 1,000 or 2,000 test images: transfer, the strongest PGD and the preprocessing defenses (2,000), and SimBA (1,000). Models are always in evaluation mode (dropout off, BatchNorm running statistics), and every random component is seeded. I report:

- **adversarial accuracy**: accuracy on the attacked images (the main robustness measure);
- **attack success rate**: among images that were correctly classified *before* the attack, the fraction misclassified afterwards (for targeted attacks: classified as the target);
- **confidence** of the wrong prediction (mean maximum softmax probability over successful attacks);
- **perceptibility**: $\|\delta\|_\infty$, $\|\delta\|_2$, the fraction of changed pixels, PSNR and SSIM (Wang et al., 2004), a similarity score in $[0, 1]$ that is closer to human judgement than pixel distances.

All computation ran on a GPU (PyTorch 2.9.1, CUDA 12.8, Python 3.11.14). A full PGD-40 attack on the 10,000 test images takes about 3.9 seconds for DigitNet.

3. Results

3.1 Clean vs. Adversarial Accuracy

Table 2 and Figure 1 compare the accuracy of the three standard models on the clean test set and under each attack. Four results stand out.

Table 2. Test accuracy (%) on all 10,000 test images, clean and under attack (L_∞ budget ε, white-box). Lower is a stronger attack.

Attack	DigitNet			SimpleCNN			TemplateDNN		
	ε=0.1	ε=0.2	ε=0.3	ε=0.1	ε=0.2	ε=0.3	ε=0.1	ε=0.2	ε=0.3
clean (no attack)		99.70			99.51			97.87	
Noise ($\pm\varepsilon$ sign)	96.0	54.5	12.2	99.4	98.9	92.5	94.0	71.1	49.4
Noise (Gaussian σ=ε)	96.1	59.9	16.4	99.4	98.9	93.0	95.5	80.9	60.3
FGSM	71.3	17.2	10.9	88.3	50.5	21.9	17.9	5.7	3.0
MI-FGSM (10 steps)	50.9	4.1	0.2	79.9	13.6	0.4	10.5	1.0	0.4
PGD-40	12.0	0.0	0.0	65.6	0.2	0.0	3.6	0.0	0.0

1. **The direction of the perturbation matters, not its size.** At $\varepsilon = 0.1$, random $\pm\varepsilon$ noise leaves DigitNet at 96.0 %,

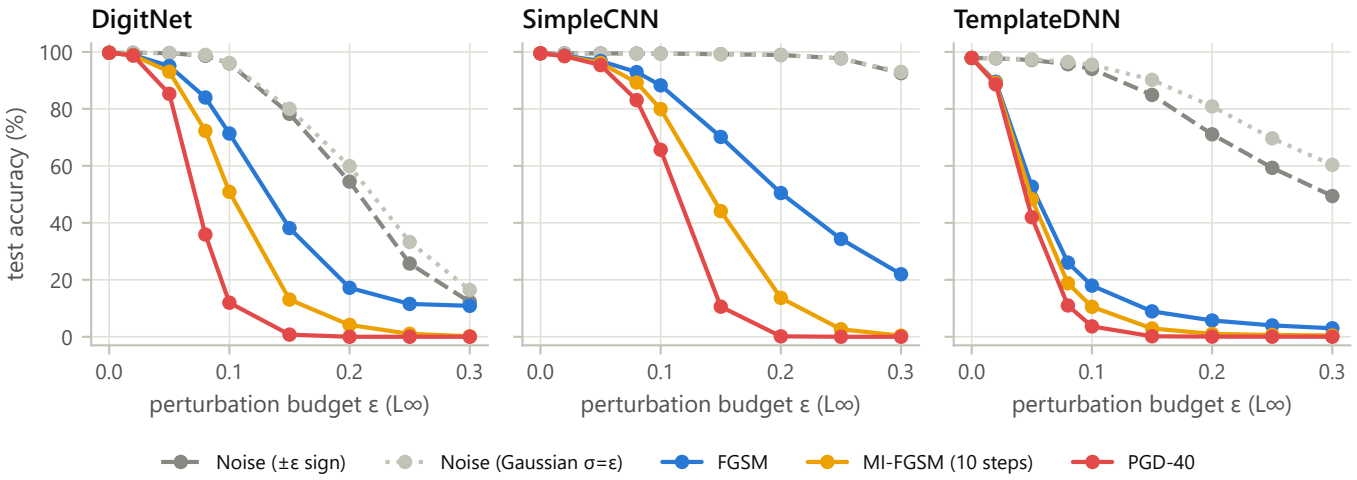


Figure 1. Test accuracy as a function of the perturbation budget ε for each model and attack. Gradient-based attacks (solid lines) destroy every model well before random noise of the same size (dashed and dotted lines) has any effect. Iterative attacks (MI-FGSM, PGD) are much stronger than the single-step FGSM.

while FGSM, which has exactly the same L_∞ and L_2 size but follows the sign of the gradient, reduces it to 71.3 %, and PGD to 12.0 %. At $\varepsilon = 0.2$ PGD succeeds on every test image (0.0 %), and the wrong predictions are made with high confidence (100.0 % on average).

2. **One gradient step underestimates the threat.** FGSM levels off at about 10.9 % for DigitNet even at $\varepsilon = 0.3$, because a single step based on a linear approximation of the loss overshoots for some images; the iterative attacks keep going to 0 %. A robustness evaluation that only uses FGSM would therefore have been far too optimistic.
3. **The most accurate model is not the most robust one.** SimpleCNN is slightly less accurate on clean data (99.51 % vs. 99.70 %), but at $\varepsilon = 0.1$ it keeps 65.6 % under PGD, against 12.0 % for DigitNet. DigitNet is even sensitive to plain random noise at larger budgets (12.2 % at $\varepsilon = 0.3$, vs. 92.5 % for SimpleCNN); Section 4.2 shows why. The fully connected TemplateDNN is the least robust model: at $\varepsilon = 0.05$, PGD already reduces it to 42.0 %.
4. **The template's results reproduce.** With the instructor's settings, TemplateDNN drops to 26.03 % under FGSM at $\varepsilon = 0.08$ (template notebook: 24.37 %), to 11.71 % under PGD at $\varepsilon = 0.08$ (11.3 %), and to 84.90 % under random noise at $\varepsilon = 0.15$ (91.81 %).

3.2 Visualizing the Perturbations

Figure 2 shows one test image of each digit next to its FGSM and PGD adversarial versions at $\varepsilon = 0.2$, and the perturbation $\delta = x' - x$ in between (red means the pixel value increased, i.e. it became darker in the displayed image; blue means it decreased). Figure 3 follows two digits as ε increases.

Three observations follow from these images:

- **The perturbation covers the whole image**, not only the digit. Most of it lies in the background: because background pixels are already 0, they can only become brighter, so the perturbation shows up there as light gray texture, while on the strokes it removes ink (blue).
- **FGSM looks like structured salt-and-pepper noise**; every pixel that can move moves by the full ε. **PGD's perturbation has visible structure**: smooth stripes and fragments of strokes, often where a stroke of the adversarial class would be (for example, closing loops that turn digits into an 8). It touches more pixels than FGSM but moves many of them by less than the full ε, so its L_2 norm is smaller (Figure 4b); it is nevertheless much more effective.
- **The wrong answers are confident.** Most PGD examples are classified with probabilities close to 100 %, so a low-confidence filter would not catch them.

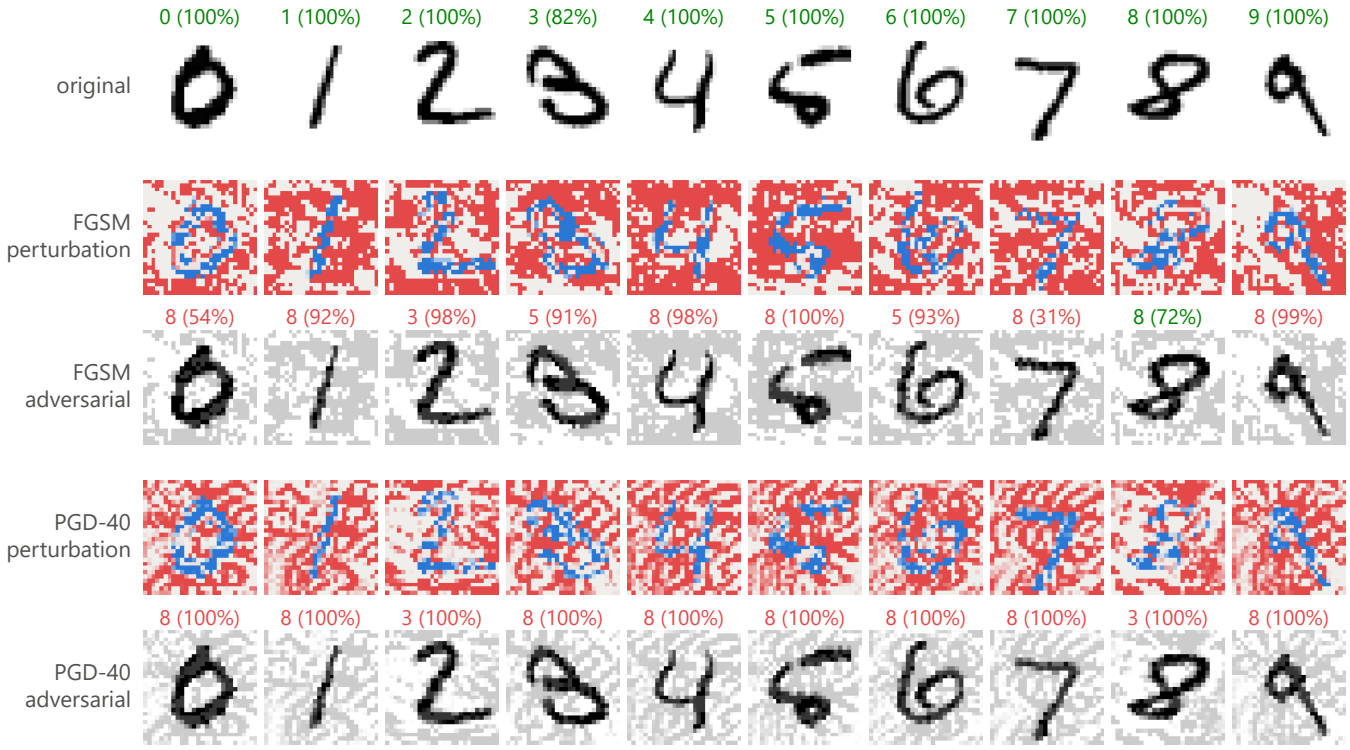


Figure 2. Original test digits (top), the FGSM and PGD perturbations at $\varepsilon = 0.2$ (red: pixel value increased, blue: decreased) and the resulting adversarial images, all classified by DigitNet. Titles give the predicted class and its softmax probability (green: correct, red: wrong). Every digit stays perfectly readable for a person, yet PGD changes all 10 predictions, each with a displayed confidence of 100 %.

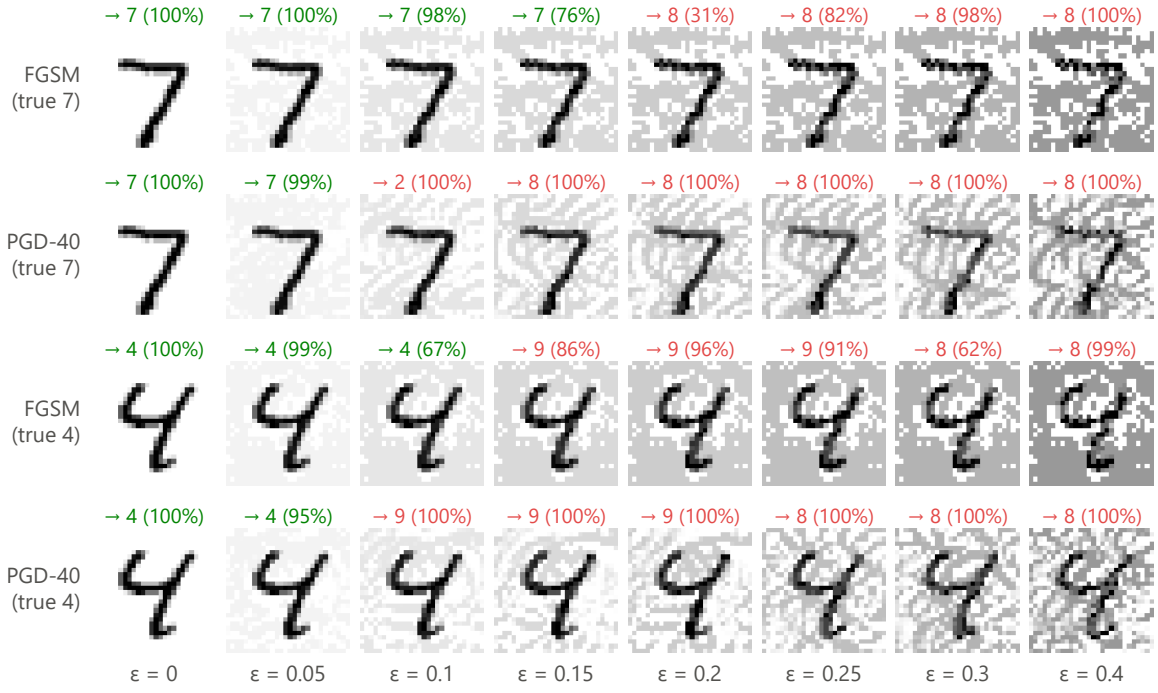


Figure 3. A 7 and a 4 attacked with growing ε. PGD flips both by $\varepsilon = 0.1$, where the change is still barely visible; FGSM needs 1.5–2 times that budget. From $\varepsilon \approx 0.25$ the perturbation becomes clearly visible as a gray texture, but the digit is still easy to read.

3.3 Effect of the Perturbation Magnitude ε

The budget ε controls a trade-off between attack success and visibility. Figure 4 measures both. Success rises steeply: PGD succeeds on 14.5 % of the images at $\varepsilon = 0.05$, 88.0 % at 0.1 and 100 % at 0.2 (FGSM: 4.7, 28.4 and 82.8 %). Visibility rises roughly linearly: the L_2 size of the PGD perturbation grows from 2.01 at $\varepsilon = 0.1$ to 5.50 at 0.3, and SSIM falls from about 0.83 at $\varepsilon = 0.02$ to about 0.44 at 0.3 for all attacks alike. SSIM is harsh on MNIST because the black background is perfectly flat, so even faint texture there counts as a large structural change; visually, the digits remain unambiguous up to about $\varepsilon = 0.3$ (Figure 3). Figure 4c combines the two: *at any level of visibility*, PGD is the most successful attack and random noise the least, so the attacker's skill matters more than the perturbation size. For MNIST, budgets around $\varepsilon = 0.1$ give the attacker almost complete success (88 % for PGD) while remaining hard to notice, which makes this range the most relevant one in practice.

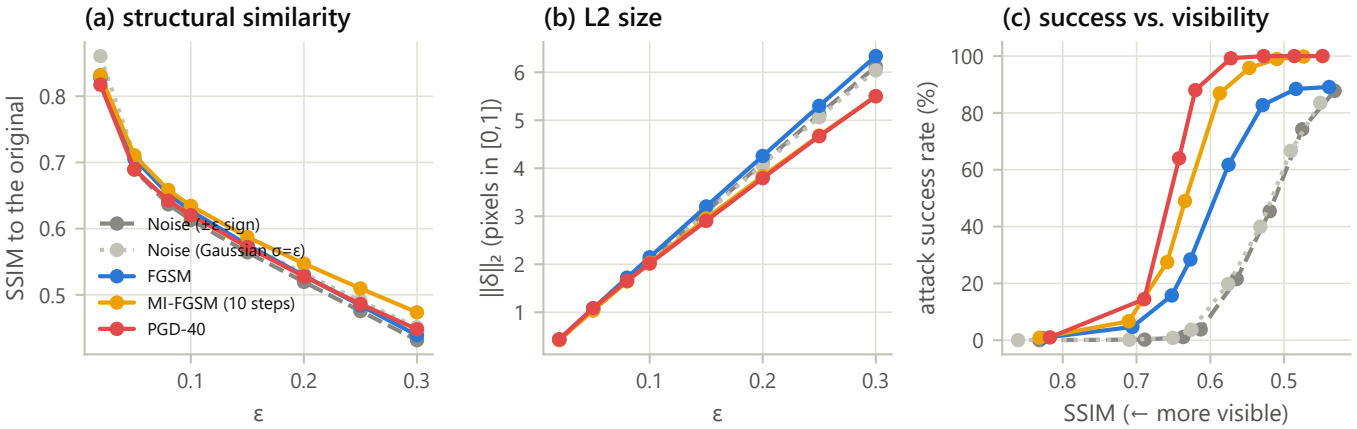


Figure 4. Visibility of the perturbation and attack success on DigitNet. (a) SSIM and (b) L_2 norm of the perturbation as functions of ε; all attacks are about equally visible for the same budget. (c) Attack success against SSIM (visibility increases to the right): at the same visibility, PGD succeeds far more often than FGSM, and both far more often than random noise.

Sanity check of the attack. A weak attack makes a model look more robust than it is, so I checked that PGD had converged. On 2,000 test images at $\varepsilon = 0.1$, accuracy falls from 54.80 % after one step to 8.95 % after 40 steps and changes little afterwards (8.60 % after 200); ten random restarts of 100 steps lower it by only about two more points (6.60 %). Accuracy also falls monotonically with ε and reaches 0 %, and white-box attacks are always stronger than black-box ones (Section 3.5), which are standard signs that the gradients are not being masked.

3.4 Where Adversarial Examples Go: Untargeted and Targeted Attacks

Figure 5a shows which wrong label each untargeted PGD example ($\varepsilon = 0.2$) receives. The answer is overwhelmingly **8**: 74.2 % of all adversarial predictions. An 8 contains most of the strokes of the other digits, so adding a few light strokes (closing loops, joining ends) is the cheapest way to leave the true class. The remaining errors follow visual similarity ($4 \to 9$, $7 \to 2$, $2 \to 3$, $8 \to 3/6$).

A **targeted** attack is harder because it has to reach one particular class. At $\varepsilon = 0.1$ targeted PGD succeeds for 50.7 % of the (source, target) pairs on average, ranging from 100.0 % for 1→0 to 0.0 % for 6→7 (Figure 5b). Again 8 is the easiest target for most source digits, and 1 and 7, which consist of few strokes, are the easiest sources. At $\varepsilon = 0.2$ the attacker can choose *any* label: the average success is 99.8 %, and even the hardest pair (8→0) succeeds 95.0 % of the time.

3.5 Black-Box Attacks

Transfer attacks (Table 3). An attacker without any access to DigitNet can train a similar model and attack that instead. Using the second DigitNet (seed 1) as the surrogate, PGD examples at $\varepsilon = 0.2$ reduce the target DigitNet to 7.3 %, almost as effective as the white-box attack. Transfer works less well across architectures but is still significant: examples crafted on SimpleCNN leave DigitNet at 26.9 %, those crafted on TemplateDNN at 56.9 %. Transfer is asymmetric: SimpleCNN's examples fool DigitNet, but DigitNet's examples hardly affect SimpleCNN (89.1 %). MI-FGSM, which

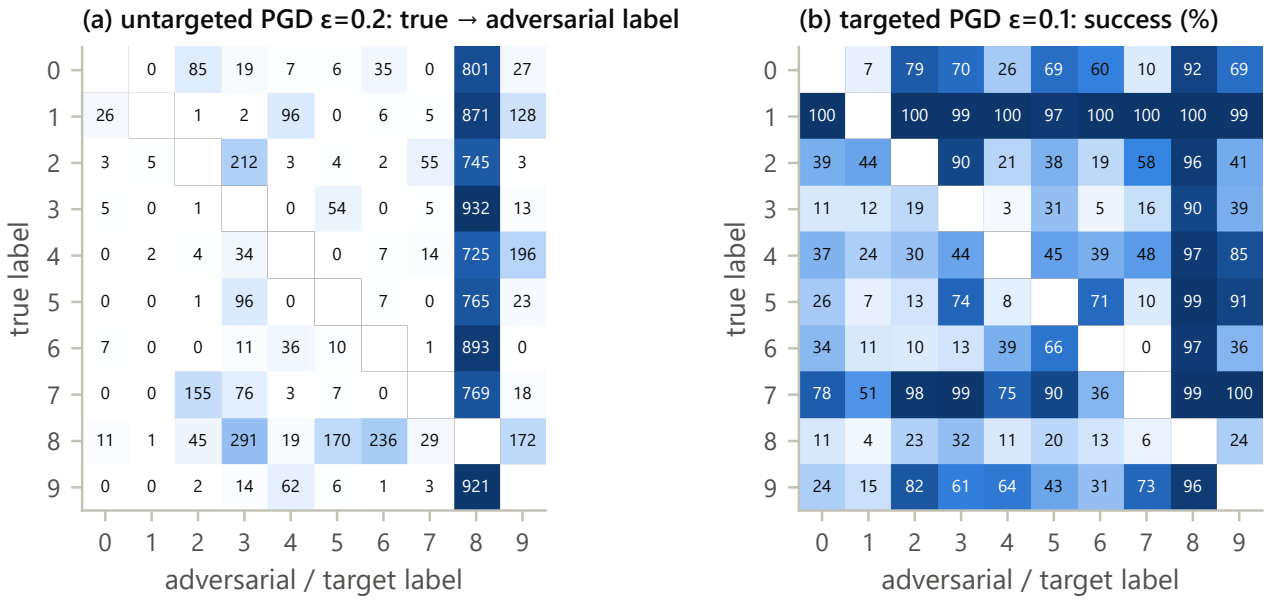


Figure 5. (a) Untargeted PGD at $\varepsilon = 0.2$: number of test images of each true digit (rows) that end up in each wrong class (columns). (b) Targeted PGD-100 at $\varepsilon = 0.1$: share of 100 correctly classified images per source digit that are classified as the chosen target.

was designed to transfer better, did not do so here (13.1 % vs. 7.3 % from the seed-1 surrogate).

Table 3. Transfer attacks (PGD-40, $\varepsilon = 0.2$, 2,000 test images): accuracy (%) of each *target* model (columns) on adversarial examples crafted on each *source* model (rows). Italic diagonal entries are white-box attacks.

Source ↓ / target →	DigitNet	DigitNet (seed 1)	SimpleCNN	TemplateDNN
DigitNet	*0.0*	7.9	89.1	60.8
DigitNet (seed 1)	7.3	*0.0*	91.0	65.8
SimpleCNN	26.9	18.9	*0.1*	47.2
TemplateDNN	56.9	30.5	93.6	*0.0*

Query-based attack. SimBA sees only the output probabilities and changes one pixel at a time. On 1,000 test images at $\varepsilon = 0.3$ it fooled DigitNet on 66.2 % of the images, with a median of 187 queries per successful attack (SimpleCNN: 51.6 %, TemplateDNN: 85.4 %). It is weaker than PGD, but it needs neither gradients nor a surrogate, only the ability to query the model a few hundred times.

3.6 Defenses

3.6.1 Adversarial training

Figure 6 shows how the two adversarially trained DigitNets developed. With **PGD adversarial training**, robust validation accuracy (PGD-20 at $\varepsilon = 0.3$) jumps once ε reaches its final value and then rises steadily to 97.7 % (epoch 19, the kept checkpoint), while clean validation accuracy stays above 99 %. **FGSM-RS training** first follows the same path, but its robust accuracy peaks at 53.9 % in epoch 7 and then collapses to 8.0 % by epoch 20, while its accuracy on its own FGSM training examples keeps rising to 99.4 %. This is *catastrophic overfitting* (Wong et al., 2020): the network learns to defeat the single FGSM step, so FGSM fails but multi-step attacks still succeed. Selecting the checkpoint by robust *validation* accuracy was essential; without it, the final FGSM-RS model would have looked robust to FGSM and been useless against PGD.

Table 4 evaluates the defended models on the full test set with white-box, strong white-box and black-box attacks.

- **PGD adversarial training works.** At $\varepsilon = 0.3$, accuracy under PGD-40 rises from 0 % to 97.3 %, and the strongest attack I ran (PGD-100 with 10 restarts) only lowers it to 95.3 %. Black-box attacks are weaker than white-box ones

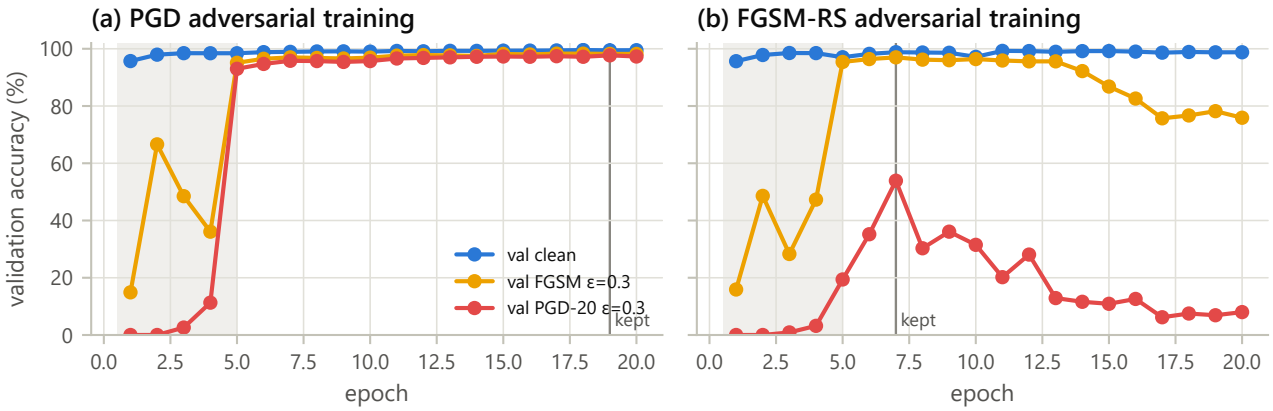


Figure 6. Validation accuracy during adversarial training at $\varepsilon = 0.3$ (1,000 validation images for the attacks). The shaded area is the ε warm-up; the vertical line marks the checkpoint that was kept (best PGD-20 accuracy). (a) PGD training converges smoothly. (b) FGSM-RS training suffers catastrophic overfitting after epoch 7: robustness to PGD collapses while clean accuracy is unaffected.

(transfer 99.1 %, SimBA 95.1 %), which is consistent with real robustness rather than gradient masking. The cost on clean data is tiny on MNIST (99.70 % → 99.49 %); the main cost is training time, about 24 seconds per epoch instead of 2.9, because every batch requires 10 extra forward and backward passes. The robustness ends at the training budget: under PGD at $\varepsilon = 0.35$ accuracy drops to 30.1 % and at 0.4 to 0.9 %.

- **FGSM-RS training (best checkpoint) is only partly robust and shows signs of gradient masking.** It resists FGSM (96.7 % at $\varepsilon = 0.3$) but keeps only 51.4 % under PGD-40 and 33.7 % under PGD with restarts, and the gradient-free SimBA attack (36.9 %) is stronger than PGD-40, a warning sign that its gradients are less informative than they should be. FGSM-RS training takes about 1.7 minutes instead of 7.9, but its robustness is much less reliable.

Table 4. Accuracy (%) of the defended models under white-box attacks at $\varepsilon \in \{0.1, 0.2, 0.3\}$ (10,000 test images), the strongest white-box attack (PGD-100 with 10 restarts, first 2,000 images), a transfer attack from the seed-1 DigitNet (PGD-40) and SimBA (first 1,000 images), all at $\varepsilon = 0.3$. For the 1-bit preprocessing defense all white-box attacks use BPDA.

Model	Clean	FGSM			PGD-40			PGD-100	Transfer	SimBA
		0.1	0.2	0.3	0.1	0.2	0.3	×10, 0.3	0.3	0.3
DigitNet (standard)	99.7	71.3	17.2	10.9	12.0	0.0	0.0	0.0	4.5	33.9
+ FGSM-RS adv. training	99.2	98.1	96.9	96.7	97.7	94.0	51.4	33.7	98.5	36.9
+ PGD adv. training	99.5	99.0	98.6	98.2	98.9	98.3	97.3	95.3	99.1	95.1
+ 1-bit input (BPDA attack)	99.4	98.1	95.8	90.6	97.7	93.2	79.8	64.0	97.7	73.0

3.6.2 Input preprocessing

I tested the three input transformations of Section 2.4 on 2,000 test images. **Median filtering and Gaussian blurring do not help**: even oblivious PGD examples crafted on the undefended model leave only 34.8 % and 56.0 % accuracy at $\varepsilon = 0.1$, and an attacker who back-propagates through the filter reaches 0 % at $\varepsilon = 0.3$. **Binarization (1-bit depth) shows how misleading a naive evaluation can be.** Against the naive white-box attacker it keeps 99.0 % at $\varepsilon = 0.3$, because rounding has zero gradient almost everywhere and PGD receives no signal; this is *gradient masking*, not robustness. With BPDA the same attack reduces it to 75.5 %, and with 10 restarts to 64.0 % (Table 4), so the naive evaluation overstated robustness by about 35 points. Binarization still helps on MNIST, whose pixels are nearly all 0 or 1 (with $\varepsilon < 0.5$ only gray edge pixels can be flipped), but this would not carry over to natural images, and PGD adversarial training remained more robust at every budget.

4. Analysis: How the Model Responds to Adversarial Inputs

4.1 Feature Maps of Clean and Adversarial Inputs

Figure 7 compares DigitNet's feature maps for a clean 4 and its PGD version ($\varepsilon = 0.2$), which is classified as an 8 with full confidence. For each stage it shows the six channels whose activation changed the most.

- **Early layers (stem, block 1)** still "see" the 4: the clean and adversarial maps both trace its strokes. The perturbation adds faint diagonal texture throughout the background, and the difference maps are spread evenly over the image rather than concentrated anywhere.
- **Block 2** (14×14) is where the two representations separate: the adversarial maps contain new blobs of activation that do not correspond to any stroke of the 4, and the difference maps become local and strong.
- **Block 3** (7×7), the last convolutional stage, is completely different: channels that were almost silent for the clean 4 fire strongly in the middle of the image, the region where an 8 has its crossing. Because global average pooling sums these maps, a few such channels are enough to move the final features towards the 8, the class most attacks end up in (Section 3.4).

4.2 The Perturbation Is Amplified Layer by Layer

To quantify this, I measured for 2,000 test images how much each layer's output changes relative to its clean value, $\|f_l(x') - f_l(x)\|_2 / \|f_l(x)\|_2$ (Figure 8). At the input, random $\pm\varepsilon$ noise and the gradient-based perturbations have the same relative size (about 0.24 at $\varepsilon = 0.1$). Through the network, however, they behave very differently:

- the **PGD perturbation grows** at every stage, from 0.23 at the input to 0.87 in the 128 final features and 0.82 in the logits. FGSM grows less (0.62 in the features);
- **random noise grows in DigitNet's early blocks too** but then shrinks again, ending at 0.40 in the features, which is not enough to change most predictions at $\varepsilon = 0.1$;
- in **SimpleCNN** random noise is damped from the start (0.24 at the input, 0.18 in the logits), while adversarial perturbations are still amplified. This is the mechanism behind the different noise robustness in Table 2: DigitNet's batch-normalized residual blocks pass even random background changes on to deeper layers, so at larger ε plain noise also starts to break it.

This is the *linearity* explanation of adversarial examples (Goodfellow et al., 2015): a perturbation that is aligned with the gradient adds up coherently over all 784 pixels and over many channels, whereas random perturbations largely cancel. A model with many high-gain directions, like DigitNet, can be very accurate and still be easy to fool.

4.3 Where the Model Looks: Grad-CAM and Saliency

Figure 9 visualizes the model's attention with Grad-CAM (Selvaraju et al., 2017) on block 3 (which regions support the predicted class) and with saliency maps (the absolute input gradient of the predicted logit, i.e. which pixels the decision is most sensitive to).

- **On clean images**, Grad-CAM covers the body of the digit, as expected from a model that has learned digit shapes.
- **On the adversarial images** (all classified as 8), the attention moves and spreads: for the 2 and the 3 it shifts towards the lower centre, for the 7 it covers a large area around the stroke, and in general it is centred where an 8 would have its loops and crossing rather than on the strokes of the true digit. The model attends to evidence that the perturbation created, not to the digit a person sees.
- **Saliency maps of the standard model** are spread over the whole image, including the empty background. This is what makes the model vulnerable: almost every pixel, even far from the strokes, has a non-zero influence on the decision, and the attacker can change all of them at once.
- **The PGD-trained model** has a very different saliency map: only a handful of pixels on or next to the strokes matter,

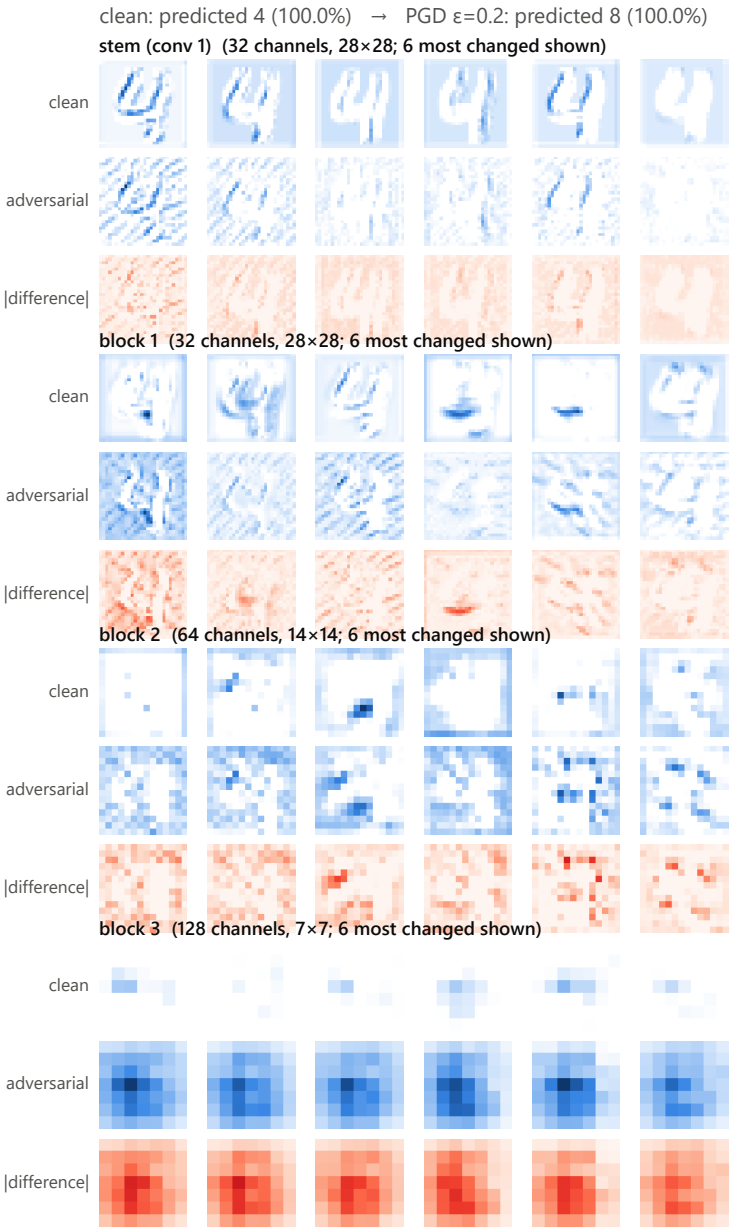


Figure 7. DigitNet's feature maps for a clean test image of a 4 ("clean") and its PGD adversarial version at $\varepsilon = 0.2$ ("adversarial"), which is classified as an 8, together with the absolute difference. Each stage shows the six channels whose activation changed most (blue: activation, red: change; the same color scale is used for clean and adversarial maps). The difference is diffuse in the early layers and becomes large and localized in blocks 2 and 3.

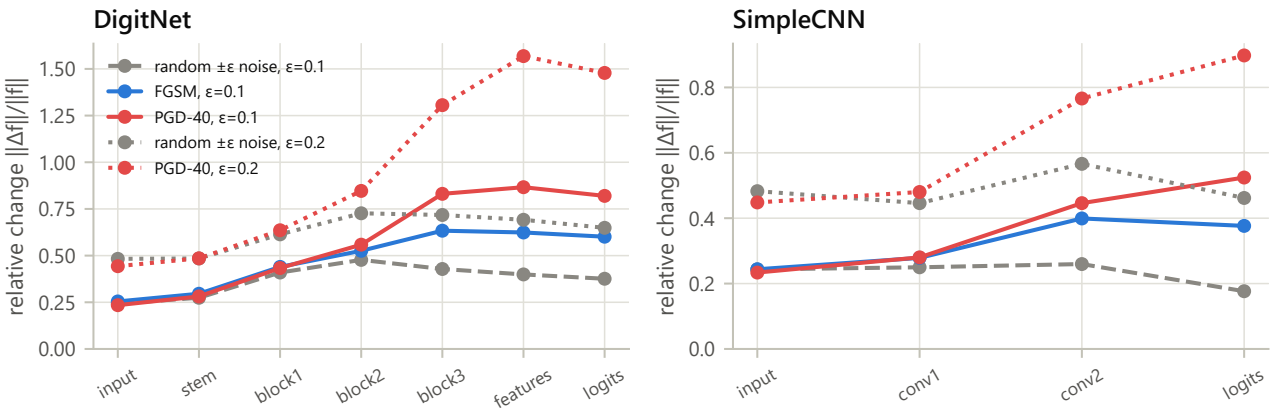


Figure 8. Relative change of each layer's output caused by random $\pm\varepsilon$ noise, FGSM and PGD perturbations of the same L_∞ size, for DigitNet and SimpleCNN (mean over 2,000 test images). Adversarial perturbations are amplified on the way to the logits; random noise is not (SimpleCNN) or much less (DigitNet).

and the background has no influence. Consistently, PGD at $\varepsilon = 0.2$ does not change any of its five predictions (last column).

4.4 Summary: Why DigitNet Is Vulnerable

Taken together, the analyses give a consistent picture. DigitNet separates the clean digits well, but it relies on many weak, distributed features: its decision depends on almost every pixel, including the background (saliency), and its layers amplify perturbations aligned with the gradient (Figure 8). An attacker who changes every pixel slightly in the right direction can therefore push the representation all the way to another class, most easily the 8, whose strokes overlap with those of most other digits. Robustness requires a model whose decisions ignore small intensity changes, and adversarial training was the most reliable way I found to obtain it.

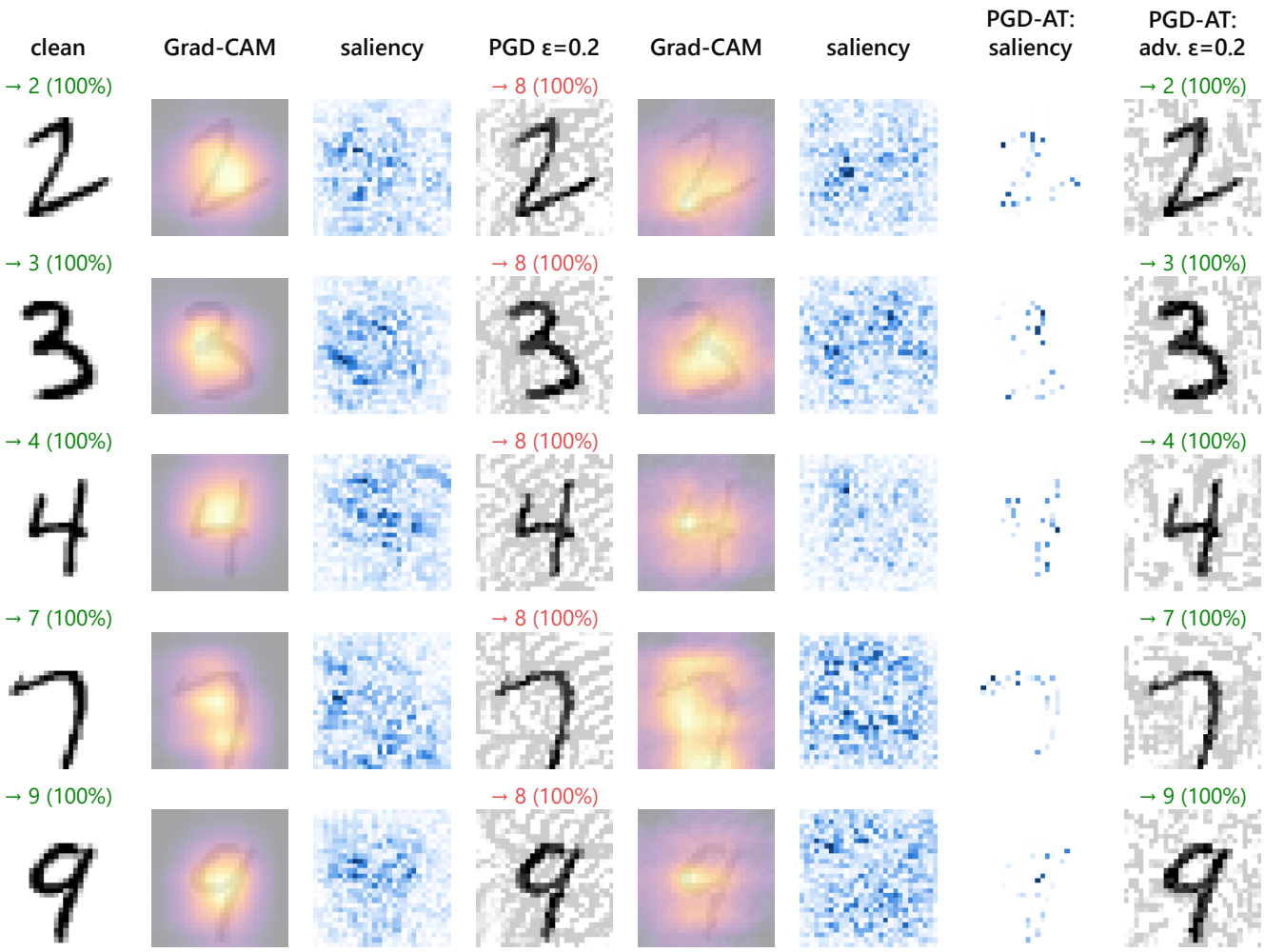


Figure 9. Attention of DigitNet on clean and adversarial images. Columns 1–3: clean image with its prediction, Grad-CAM heat map of the predicted class (bright = important) and input-gradient saliency. Columns 4–6: the same for the PGD adversarial image ($\varepsilon = 0.2$), which DigitNet classifies as 8. Columns 7–8: saliency of the PGD-trained DigitNet on the clean image, and a PGD attack ($\varepsilon = 0.2$) against that model, which fails.

5. Conclusions and Recommendations

5.1 Conclusions

1. **High accuracy does not imply robustness.** DigitNet classifies 99.70 % of the clean test images correctly, yet PGD with $\varepsilon = 0.1$ leaves 12.0 % and with $\varepsilon = 0.2$ none. The adversarial predictions are made with near-certain confidence, although every digit remains easy to read. The more accurate DigitNet was *less* robust than the simpler SimpleCNN.
2. **The attack matters as much as the budget.** Random noise is harmless at $\varepsilon = 0.1$, FGSM is moderately effective and PGD is devastating, although all three perturbations are equally large and equally visible. Robustness must therefore be evaluated with strong, iterative and adaptive attacks; single-step FGSM results are optimistic.
3. **Inside the network,** adversarial perturbations are amplified layer by layer until they dominate the final features, while random noise is not. Attention maps shift towards the regions the attacker modified, and the adversarial image is classified as another class with near-certain confidence, most often 8, the digit that can be formed by adding a few light strokes.
4. **Attacks transfer.** An attacker without access to the model can craft adversarial examples on a self-trained surrogate (7.3 % accuracy left on DigitNet) or by querying the model a few hundred times (SimBA).
5. **Of the defenses, PGD adversarial training was by far the most effective.** PGD adversarial training raised accuracy

under PGD at $\varepsilon = 0.3$ from 0 % to 97.3 % (95.3 % under 100-step PGD with 10 restarts) at a clean-accuracy cost of only 99.70 % → 99.49 %, but training took 7.9 instead of 1.9 minutes, and robustness disappears just beyond the training budget. Median filtering and blurring gave no protection. Binarization appeared almost perfectly robust when evaluated naively, but that was gradient masking: an adaptive BPDA attacker reduced it to 64.0 % at $\varepsilon = 0.3$. It remains a useful extra layer on MNIST, whose pixels are nearly binary, but it is weaker than adversarial training.

5.2 Recommendations for Improving Robustness

Countermeasures.

- **Use PGD adversarial training as the primary defense**, with an ε at least as large as the realistic threat and an ε ramp-up at the start of training. If single-step FGSM training is used for speed, monitor robust validation accuracy under PGD to catch catastrophic overfitting (Wong et al., 2020).
- **Evaluate with adaptive attacks**: many-step PGD with restarts, BPDA for non-differentiable components, and black-box attacks as a sanity check (Athalye et al., 2018; Carlini and Wagner, 2017). Robustness reported only under the attack a defense was designed for is misleading.
- **Reduce the black-box attack surface and add detection as an extra layer**: return only the top label, rate-limit queries (SimBA needed a few hundred) and keep the weights confidential; detectors such as feature squeezing (Xu et al., 2018) cheaply flag non-adaptive adversarial examples, but cannot replace adversarial training.

Architectural changes.

- **Control the network's gain.** The amplification analysis (Section 4.2) suggests limiting how much each layer amplifies small input changes, e.g. with spectral normalization, Lipschitz-constrained layers or an input-gradient penalty.
- **Add capacity and diversity.** Robust models need more capacity than standard ones (Madry et al., 2018), so wider blocks should accompany adversarial training; and because transfer between DigitNet and SimpleCNN was incomplete (Table 3), an ensemble of different adversarially trained architectures would force an attacker to fool all of them at once.

Preprocessing steps.

- **Binarize MNIST inputs, as defense in depth.** Handwritten digits are essentially black and white, so 1-bit depth costs little clean accuracy (99.15 %) and stops cheap, non-adaptive and transferred attacks; but it lost about 35 points of apparent robustness against an adaptive attacker, while smoothing filters were bypassed completely (Section 3.6.2).
- **Randomized preprocessing**, such as random resizing, padding or noise, makes the attacker's gradient less reliable and, as randomized smoothing (Cohen et al., 2019), can even give a certified radius within which the prediction cannot change; it must be evaluated with expectation-over-transformation attacks.

6. Appendix — Source Code

- **GitHub repository:** `https://github.com/EBARV/CYSE635-MNIST-Adversarial`.
  It contains this report and the notebook `Module03-Adversarial-Examples.ipynb`, which holds all of the source code (models, attacks, defenses, metrics and plots) and every experiment in this report, with its tables and figures shown as outputs. It ran from top to bottom in about 7 minutes on the GPU (plus about 7.9 and 1.7 minutes to train the two adversarially trained models). It loads the DigitNet and SimpleCNN checkpoints produced by the Module 02 notebook.
- **Code snippet.** The core of the lab, the FGSM and PGD attacks, is reproduced in Listing 1 with its comments; the model and the remaining code are in the notebook; the BPDA wrapper is a one-line change of the forward pass, $t = x + (\text{transform}(x) - x)$`.detach()`.
- **Graphs and visualizations.** All figures are included with captions in the main text.

Listing 1. FGSM and PGD (from the project notebook). All attacks share the helpers `input_grad` and `project`.

```python
def input_grad(model, x, y):
    """Gradient of the summed cross-entropy loss w.r.t. the input (one forward + backward pass).
    reduction='sum' keeps every image's gradient independent of the batch size."""
    with torch.enable_grad():
        x = x.detach().requires_grad_(True)
        logits = model(x)
        loss = F.cross_entropy(logits, y, reduction="sum")
        (g,) = torch.autograd.grad(loss, x)
    return g, logits.detach()

def project(x_adv, x, eps):
    """Projection onto the allowed set: the L_inf ball of radius eps around x, intersected with [0, 1]."""
    return torch.min(torch.max(x_adv, x - eps), x + eps).clamp(0, 1)

def fgsm(model, x, y, eps, targeted=False):
    """Fast Gradient Sign Method (Goodfellow et al., 2015): x' = clip(x + eps * sign(grad_x J))."""
    g, _ = input_grad(model, x, y)
    step = _eps(eps, x) * g.sign()
    return (x - step if targeted else x + step).clamp(0, 1).detach()

def pgd(model, x, y, eps, alpha=None, steps=40, random_start=True, restarts=1, targeted=False):
    """Projected Gradient Descent (Madry et al., 2018): x_{t+1} = Proj_eps(x_t + alpha * sign(grad)),
    from a random start. Per image, the first successful (else highest-loss) restart is kept."""
    e = _eps(eps, x)
    a = 2.5 * e / steps if alpha is None else _eps(alpha, x)
    best, done = x.clone(), torch.zeros(len(x), dtype=torch.bool, device=x.device)
    best_obj = torch.full((len(x),), -float("inf"), device=x.device)
    for _ in range(restarts):
        x_adv = project(x + (torch.rand_like(x) * 2 - 1) * e, x, e) if random_start else x.clone()
        for _ in range(steps):
            g, _ = input_grad(model, x_adv, y)
            x_adv = project(x_adv - a * g.sign() if targeted else x_adv + a * g.sign(), x, e)
        with torch.no_grad():
            logits = model(x_adv)
            ce = F.cross_entropy(logits, y, reduction="none")
            succ = (logits.argmax(1) == y) if targeted else (logits.argmax(1) != y)
            obj = -ce if targeted else ce
            better = (succ & ~done) | ((succ == done) & (obj > best_obj))
            best[better], best_obj[better] = x_adv[better], obj[better]
            done |= succ
    return best.detach()
```